JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ResourceBundle

java.lang.Object
    java.util.ResourceBundle

Direct Known Subclasses:

ListResourceBundle, PropertyResourceBundle

```
public abstract class ResourceBundle
extends Object
```

Resource bundles contain locale-specific objects. When your program needs a locale-specific resource, a `String` for example, your program can load it from the resource bundle that is appropriate for the current user's locale. In this way, you can write program code that is largely independent of the user's locale isolating most, if not all, of the locale-specific information in resource bundles.

This allows you to write programs that can:

- be easily localized, or translated, into different languages
- handle multiple locales at once
- be easily modified later to support even more locales

Resource bundles belong to families whose members share a common base name, but whose names also have additional components that identify their locales. For example, the base name of a family of resource bundles might be "MyResources". The family should have a default resource bundle which simply has the same name as its family - "MyResources" - and will be used as the bundle of last resort if a specific locale is not supported. The family can then provide as many locale-specific members as needed, for example a German one named "MyResources_de".

Each resource bundle in a family contains the same items, but the items have been translated for the locale represented by that resource bundle. For example, both "MyResources" and "MyResources_de" may have a `String` that's used on a button for canceling operations. In "MyResources" the `String` may contain "Cancel" and in "MyResources_de" it may contain "Abbrechen".

If there are different resources for different countries, you can make specializations: for example, "MyResources_de_CH" contains objects for the German language (de) in Switzerland (CH). If you want to only modify some of the resources in the specialization, you can do so.

When your program needs a locale-specific object, it loads the `ResourceBundle` class using the `getBundle` method:

```
ResourceBundle myResources =
     ResourceBundle.getBundle("MyResources", currentLocale);
```

Resource bundles contain key/value pairs. The keys uniquely identify a locale-specific object in the bundle. Here's an example of a `ListResourceBundle` that contains two key/value pairs:

```
public class MyResources extends ListResourceBundle {
    protected Object[][] getContents() {
        return new Object[][] {
            // LOCALIZE THE SECOND STRING OF EACH ARRAY (e.g., "OK")
            {"OkKey", "OK"},
            {"CancelKey", "Cancel"},
            // END OF MATERIAL TO LOCALIZE
       };
    }
}
```

Keys are always `Strings`. In this example, the keys are "OkKey" and "CancelKey". In the above example, the values are also `Strings`--"OK" and "Cancel"--but they don't have to be. The values can be any type of object.

You retrieve an object from resource bundle using the appropriate getter method. Because "OkKey" and "CancelKey" are both strings, you would use `getString` to retrieve them:

```
button1 = new Button(myResources.getString("OkKey"));
button2 = new Button(myResources.getString("CancelKey"));
```

The getter methods all require the key as an argument and return the object if found. If the object is not found, the getter method throws a `MissingResourceException`.

Besides `getString`, `ResourceBundle` also provides a method for getting string arrays, `getStringArray`, as well as a generic `getObject` method for any other type of object. When using `getObject`, you'll have to cast the result to the appropriate type. For example:

```
int[] myIntegers = (int[]) myResources.getObject("intList");
```

The Java Platform provides two subclasses of `ResourceBundle`, `ListResourceBundle` and `PropertyResourceBundle`, that provide a fairly simple way to create resources. As you saw briefly in a previous example,

`ListResourceBundle` manages its resource as a list of key/value pairs. `PropertyResourceBundle` uses a properties file to manage its resources.

If `ListResourceBundle` or `PropertyResourceBundle` do not suit your needs, you can write your own `ResourceBundle` subclass. Your subclasses must override two methods: `handleGetObject` and `getKeys()`.

The implementation of a `ResourceBundle` subclass must be thread-safe if it's simultaneously used by multiple threads. The default implementations of the non-abstract methods in this class, and the methods in the direct known concrete subclasses `ListResourceBundle` and `PropertyResourceBundle` are thread-safe.

Resource Bundles and Named Modules

Resource bundles can be deployed in modules in the following ways:

Resource bundles together with an application

Resource bundles can be deployed together with an application in the same module. In that case, the resource bundles are loaded by code in the module by calling the `getBundle(String)` or `getBundle(String, Locale)` method.

Resource bundles as service providers

Resource bundles can be deployed in one or more *service provider modules* and they can be located using `ServiceLoader`. A service interface or class must be defined. The caller module declares that it uses the service, the service provider modules declare that they provide implementations of the service. Refer to `ResourceBundleProvider` for developing resource bundle services and deploying resource bundle providers. The module obtaining the resource bundle can be a resource bundle provider itself; in which case this module only locates the resource bundle via service provider mechanism.

A resource bundle provider can provide resource bundles in any format such XML which replaces the need of `ResourceBundle.Control`.

Resource bundles in other modules and class path

Resource bundles in a named module may be *encapsulated* so that it cannot be located by code in other modules. Resource bundles in unnamed modules and class path are open for any module to access. Resource bundle follows the resource encapsulation rules as specified in `Module.getResourceAsStream(String)`.

The `getBundle` factory methods with no `Control` parameter locate and load resource bundles from service providers. It may continue the search as if

calling `Module.getResourceAsStream(String)` to find the named resource from a given module and calling `ClassLoader.getResourceAsStream(String)`; refer to the specification of the `getBundle` method for details. Only non-encapsulated resource bundles of "`java.class`" or "`java.properties`" format are searched.

If the caller module is a resource bundle provider, it does not fall back to the class loader search.

In cases where the `getBundle` factory method is called from a context where there is no caller frame on the stack (e.g. when called directly from a JNI attached thread), the caller module is default to the unnamed module for the system class loader.

Resource bundles in automatic modules

A common format of resource bundles is in .properties file format. Typically `.properties` resource bundles are packaged in a JAR file. Resource bundle only JAR file can be readily deployed as an automatic module. For example, if the JAR file contains the entry "`p/q/Foo_ja.properties`" and no `.class` entry, when resolved and defined as an automatic module, no package
is derived for this module. This allows resource bundles in `.properties` format packaged in one or more JAR files that may contain entries in the same directory and can be resolved successfully as automatic modules.

ResourceBundle.Control

The `ResourceBundle.Control` class provides information necessary to perform the bundle loading process by the `getBundle` factory methods that take a `ResourceBundle.Control` instance. You can implement your own subclass in order to enable non-standard resource bundle formats, change the search strategy, or define caching parameters. Refer to the descriptions of the class and the `getBundle` factory method for details.

`ResourceBundle.Control` is designed for an application deployed in an unnamed module, for example to support resource bundles in non-standard formats or package localized resources in a non-traditional convention. `ResourceBundleProvider` is the replacement for `ResourceBundle.Control` when migrating to modules. `UnsupportedOperationException` will be thrown when a factory method that takes the `ResourceBundle.Control` parameter is called.

For the `getBundle` factory methods that take no `ResourceBundle.Control` instance, their default behavior of resource bundle loading can be modified with custom `ResourceBundleControlProvider` implementations. If any of the providers provides a `ResourceBundle.Control` for the given base name, that `ResourceBundle.Control` will be used instead of the default

`ResourceBundle.Control`. If there is more than one service provider for supporting the same base name, the first one returned from `ServiceLoader` will be used. A custom `ResourceBundle.Control` implementation is ignored by named modules.

Cache Management

Resource bundle instances created by the `getBundle` factory methods are cached by default, and the factory methods return the same resource bundle instance multiple times if it has been cached. `getBundle` clients may clear the cache, manage the lifetime of cached resource bundle instances using time-to-live values, or specify not to cache resource bundle instances. Refer to the descriptions of the `getBundle` factory method, `clearCache`, `ResourceBundle.Control.getTimeToLive`, and `ResourceBundle.Control.needsReload` for details.

Example

The following is a very simple example of a `ResourceBundle` subclass, `MyResources`, that manages two resources (for a larger number of resources you would probably use a `Map`). Notice that you don't need to supply a value if a "parent-level" `ResourceBundle` handles the same key with the same value (as for the okKey below).

```
// default (English language, United States)
public class MyResources extends ResourceBundle {
    public Object handleGetObject(String key) {
        if (key.equals("okKey")) {
           return "Ok";
        }
        if (key.equals("cancelKey")) {
           return "Cancel";
        }
        return null;
    }

    public Enumeration<String> getKeys() {
        return Collections.enumeration(keySet());
    }

    // Overrides handleKeySet() so that the getKeys() implementation
    // can rely on the keySet() value.
    protected Set<String> handleKeySet() {
        return new HashSet<String>(Arrays.asList("okKey", "cancelKey"));
```

```
    }
}

// German language
public class MyResources_de extends MyResources {
    public Object handleGetObject(String key) {
        // don't need okKey, since parent level handles it.
        if (key.equals("cancelKey")) {
           return "Abbrechen";
        }
        return null;
    }

    protected Set<String> handleKeySet() {
        return new HashSet<String>(Arrays.asList("cancelKey"));
    }
}
```

You do not have to restrict yourself to using a single family of `ResourceBundles`. For example, you could have a set of bundles for exception messages, `ExceptionResources` (`ExceptionResources_fr`, `ExceptionResources_de`, ...), and one for widgets, `WidgetResource` (`WidgetResources_fr`, `WidgetResources_de`, ...); breaking up the resources however you like.

Since:

1.1

See Also:

`ListResourceBundle`, `PropertyResourceBundle`, `MissingResourceException`, `ResourceBundleProvider`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static class`	**`ResourceBundle.Control`**	`ResourceBundle.Control` defines a set of callback methods that are invoked by the `ResourceBundle.getBundle` factory methods during

	the bundle loading process.

Field Summary

Fields

Modifier and Type	Field	Description
protected ResourceBundle	parent	The parent bundle of this bundle.

Constructor Summary

Constructors

Constructor	Description
ResourceBundle()	Sole constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
static final void	clearCache()	Removes all resource bundles from the cache that have been loaded by the caller's module.
static final void	clearCache (ClassLoader loader)	Removes all resource bundles from the cache that have been loaded by the given class loader.
boolean	containsKey(String key)	Determines whether the given key is contained in

		this ResourceBundle or its parent bundles.
String	**getBaseBundleName**()	Returns the base name of this bundle, if known, or null if unknown.
static final ResourceBundle	**getBundle** (String baseName)	Gets a resource bundle using the specified base name, the default locale, and the caller module.
static ResourceBundle	**getBundle** (String baseName, Module module)	Gets a resource bundle using the specified base name and the default locale on behalf of the specified module.
static final ResourceBundle	**getBundle** (String baseName, Locale locale)	Gets a resource bundle using the specified base name and locale, and the caller module.
static ResourceBundle	**getBundle** (String baseName, Locale locale, ClassLoader loader)	Gets a resource bundle using the specified base name, locale, and class loader.
static ResourceBundle	**getBundle** (String baseName, Locale targetLocale, ClassLoader loader, ResourceBundle.Control c	Returns a resource bundle using the specified base name, target locale, class loader and control.
static ResourceBundle	**getBundle** (String baseName, Locale targetLocale, Module module)	Gets a resource bundle using the specified base name and locale on behalf of the specified module.
static final ResourceBundle	**getBundle** (String baseName, Locale targetLocale, ResourceBundle.Control c	Returns a resource bundle using the specified base name, target locale and

		control, and the caller's class loader.
static final ResourceBundle	**getBundle**(String baseName, ResourceBundle.Control c	Returns a resource bundle using the specified base name, the default locale and the specified control.
abstract Enumeration<String>	**getKeys**()	Returns an enumeration of the keys.
Locale	**getLocale**()	Returns the locale of this resource bundle.
final Object	**getObject**(String key)	Gets an object for the given key from this resource bundle or one of its parents.
final String	**getString**(String key)	Gets a string for the given key from this resource bundle or one of its parents.
final String[]	**getStringArray**(String key)	Gets a string array for the given key from this resource bundle or one of its parents.
protected abstract Object	**handleGetObject**(String key)	Gets an object for the given key from this resource bundle.
protected Set<String>	**handleKeySet**()	Returns a `Set` of the keys contained *only* in this `ResourceBundle`.
Set<String>	**keySet**()	Returns a `Set` of all keys contained in this `ResourceBundle` and its parent bundles.

protected void	**setParent**(ResourceBundle parent)	Sets the parent bundle of this bundle.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

parent

protected ResourceBundle parent

The parent bundle of this bundle. The parent bundle is searched by getObject when this bundle does not contain a particular resource.

Constructor Details

ResourceBundle

public ResourceBundle()

Sole constructor. (For invocation by subclass constructors, typically implicit.)

Method Details

getBaseBundleName

public String getBaseBundleName()

Returns the base name of this bundle, if known, or null if unknown. If not null, then this is the value of the baseName parameter that was passed to the ResourceBundle.getBundle(...) method when the resource bundle was loaded.

Returns:

The base name of the resource bundle, as provided to and expected by the ResourceBundle.getBundle(...) methods.

Since:

1.8

See Also:

getBundle(java.lang.String, java.util.Locale, java.lang.ClassLoader)

getString

```
public final String getString(String key)
```

Gets a string for the given key from this resource bundle or one of its parents. Calling this method is equivalent to calling

```
    (String) getObject(key);
```

Parameters:

key - the key for the desired string

Returns:

the string for the given key

Throws:

NullPointerException - if key is null

MissingResourceException - if no object for the given key can be found

ClassCastException - if the object found for the given key is not a string

getStringArray

```
public final String[] getStringArray(String key)
```

Gets a string array for the given key from this resource bundle or one of its parents. Calling this method is equivalent to calling

```
    (String[]) getObject(key);
```

Parameters:

key - the key for the desired string array

Returns:

the string array for the given key

Throws:

NullPointerException - if key is null

MissingResourceException - if no object for the given key can be found

ClassCastException - if the object found for the given key is not a string array

getObject

```
public final Object getObject(String key)
```

Gets an object for the given key from this resource bundle or one of its parents. This method first tries to obtain the object from this resource bundle using `handleGetObject`. If not successful, and the parent resource bundle is not null, it calls the parent's `getObject` method. If still not successful, it throws a MissingResourceException.

Parameters:

`key` - the key for the desired object

Returns:

the object for the given key

Throws:

`NullPointerException` - if `key` is `null`

`MissingResourceException` - if no object for the given key can be found

getLocale

```
public Locale getLocale()
```

Returns the locale of this resource bundle. This method can be used after a call to getBundle() to determine whether the resource bundle returned really corresponds to the requested locale or is a fallback.

Returns:

the locale of this resource bundle

setParent

```
protected void setParent(ResourceBundle parent)
```

Sets the parent bundle of this bundle. The parent bundle is searched by `getObject` when this bundle does not contain a particular resource.

Parameters:

`parent` - this bundle's parent bundle.

getBundle

```
public static final ResourceBundle getBundle(String baseName)
```

Gets a resource bundle using the specified base name, the default locale, and the caller module. Calling this method is equivalent to calling

```
    getBundle(baseName, Locale.getDefault(), callerModule);
```

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

Returns:

a resource bundle for the given base name and the default locale

Throws:

`NullPointerException` - if `baseName` is `null`

`MissingResourceException` - if no resource bundle for the specified base name can be found

See Also:

Resource Bundle Search and Loading Strategy,
Resource Bundles and Named Modules

getBundle

```
public static final ResourceBundle getBundle
(String baseName,
 ResourceBundle.Control control)
```

Returns a resource bundle using the specified base name, the default locale and the specified control. Calling this method is equivalent to calling

```
getBundle(baseName, Locale.getDefault(),
          this.getClass().getClassLoader(), control);
```

except that `getClassLoader()` is run with the security privileges of `ResourceBundle`. See `getBundle` for the complete description of the resource bundle loading process with a `ResourceBundle.Control`.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`control` - the control which gives information for the resource bundle loading process

Returns:

a resource bundle for the given base name and the default locale

Throws:

`NullPointerException` - if `baseName` or `control` is `null`

`MissingResourceException` - if no resource bundle for the specified base name can be found

IllegalArgumentException - if the given control doesn't perform properly (e.g., control.getCandidateLocales returns null.) Note that validation of control is performed as needed.

UnsupportedOperationException - if this method is called in a named module

Since:

1.6

getBundle

```
public static final ResourceBundle getBundle(String baseName,
                                             Locale locale)
```

Gets a resource bundle using the specified base name and locale, and the caller module. Calling this method is equivalent to calling

```
    getBundle(baseName, locale, callerModule);
```

Parameters:

baseName - the base name of the resource bundle, a fully qualified class name

locale - the locale for which a resource bundle is desired

Returns:

a resource bundle for the given base name and locale

Throws:

NullPointerException - if baseName or locale is null

MissingResourceException - if no resource bundle for the specified base name can be found

See Also:

Resource Bundle Search and Loading Strategy,
Resource Bundles and Named Modules

getBundle

```
public static ResourceBundle getBundle(String baseName,
                                       Module module)
```

Gets a resource bundle using the specified base name and the default locale on behalf of the specified module. This method is equivalent to calling

```
    getBundle(baseName, Locale.getDefault(), module);
```

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`module` - the module for which the resource bundle is searched

Returns:

a resource bundle for the given base name and the default locale

Throws:

`NullPointerException` - if `baseName` or `module` is `null`

`MissingResourceException` - if no resource bundle for the specified base name can be found in the specified module

Since:

9

See Also:

`ResourceBundleProvider`,
Resource Bundle Search and Loading Strategy,
Resource Bundles and Named Modules

getBundle

```
public static ResourceBundle getBundle(String baseName,
                                       Locale targetLocale,
                                       Module module)
```

Gets a resource bundle using the specified base name and locale on behalf of the specified module.

Resource bundles in named modules may be encapsulated. When the resource bundle is loaded from a service provider, the caller module must have an appropriate *uses* clause in its *module descriptor* to declare that the module uses of `ResourceBundleProvider` for the named resource bundle. Otherwise, it
will load the resource bundles that are local in the given module as if calling `Module.getResourceAsStream(String)` or that are visible to the class loader of the given module as if calling `ClassLoader.getResourceAsStream(String)`. When the resource bundle is loaded from the specified module, it is subject to the encapsulation rules specified by `Module.getResourceAsStream`.

If the given `module` is an unnamed module, then this method is equivalent to calling `getBundle(baseName, targetLocale, module.getClassLoader()` to load resource bundles that are visible to the class loader of the given unnamed module. Custom `ResourceBundleControlProvider` implementations, if present, will only be invoked if the specified module is an unnamed module.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`targetLocale` - the locale for which a resource bundle is desired

`module` - the module for which the resource bundle is searched

Returns:

a resource bundle for the given base name and locale in the module

Throws:

`NullPointerException` - if `baseName`, `targetLocale`, or `module` is `null`

`MissingResourceException` - if no resource bundle for the specified base name and locale can be found in the specified `module`

Since:

9

See Also:

Resource Bundle Search and Loading Strategy,
Resource Bundles and Named Modules

getBundle

```
public static final ResourceBundle getBundle
(String baseName,
 Locale targetLocale,
 ResourceBundle.Control control)
```

Returns a resource bundle using the specified base name, target locale and control, and the caller's class loader. Calling this method is equivalent to calling

```
getBundle(baseName, targetLocale, this.getClass().getClassLoader(),
          control);
```

except that `getClassLoader()` is run with the security privileges of `ResourceBundle`. See `getBundle` for the complete description of the resource bundle loading process with a `ResourceBundle.Control`.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`targetLocale` - the locale for which a resource bundle is desired

`control` - the control which gives information for the resource bundle loading process

Returns:

a resource bundle for the given base name and a `Locale` in `locales`

Throws:

`NullPointerException` - if `baseName`, `locales` or `control` is `null`

`MissingResourceException` - if no resource bundle for the specified base name in any of the `locales` can be found.

`IllegalArgumentException` - if the given `control` doesn't perform properly (e.g., `control.getCandidateLocales` returns null.) Note that validation of `control` is performed as needed.

`UnsupportedOperationException` - if this method is called in a named module

Since:

1.6

getBundle

```
public static ResourceBundle getBundle(String baseName,
                                       Locale locale,
                                       ClassLoader loader)
```

Gets a resource bundle using the specified base name, locale, and class loader.

When this method is called from a named module and the given loader is the class loader of the caller module, this is equivalent to calling:

```
    getBundle(baseName, targetLocale, callerModule);
```

otherwise, this is equivalent to calling:

```
    getBundle(baseName, targetLocale, loader, control);
```

where `control` is the default instance of `ResourceBundle.Control` unless a `Control` instance is provided by `ResourceBundleControlProvider` SPI. Refer to the description of modifying the default behavior. The following describes the default behavior.

Resource Bundle Search and Loading Strategy

`getBundle` uses the base name, the specified locale, and the default locale (obtained from `Locale.getDefault`) to generate a sequence of *candidate bundle names*. If the specified locale's language, script, country, and variant are all empty strings, then the base name is the only candidate bundle name.

Otherwise, a list of candidate locales is generated from the attribute values of the specified locale (language, script, country and variant) and appended to the base name. Typically, this will look like the following:

```
    baseName + "_" + language + "_" + script + "_" + country + "_" + variant
    baseName + "_" + language + "_" + script + "_" + country
    baseName + "_" + language + "_" + script
    baseName + "_" + language + "_" + country + "_" + variant
    baseName + "_" + language + "_" + country
    baseName + "_" + language
```

Candidate bundle names where the final component is an empty string are omitted, along with the underscore. For example, if country is an empty string, the second and the fifth candidate bundle names above would be omitted. Also, if script is an empty string, the candidate names including script are omitted. For example, a locale with language "de" and variant "JAVA" will produce candidate names with base name "MyResource" below.

```
    MyResource_de__JAVA
    MyResource_de
```

In the case that the variant contains one or more underscores ('_'), a sequence of bundle names generated by truncating the last underscore and the part following it is inserted after a candidate bundle name with the original variant. For example, for a locale with language "en", script "Latn", country "US" and variant "WINDOWS_WIN11", and bundle base name "MyResource", the list of candidate bundle names below is generated:

```
MyResource_en_Latn_US_WINDOWS_WIN11
MyResource_en_Latn_US_WINDOWS
MyResource_en_Latn_US
MyResource_en_Latn
MyResource_en_US_WINDOWS_WIN11
MyResource_en_US_WINDOWS
MyResource_en_US
MyResource_en
```

> **Note:** For some `Locales`, the list of candidate bundle names contains extra names, or the order of bundle names is slightly modified. See the description of the default implementation of `getCandidateLocales` for details.

`getBundle` then iterates over the candidate bundle names to find the first one for which it can *instantiate* an actual resource bundle. It uses the default

controls' `getFormats` method, which generates two bundle names for each generated name, the first a class name and the second a properties file name. For each candidate bundle name, it attempts to create a resource bundle:

- First, it attempts to load a class using the generated class name. If such a class can be found and loaded using the specified class loader, is assignment compatible with ResourceBundle, is accessible from ResourceBundle, and can be instantiated, `getBundle` creates a new instance of this class and uses it as the *result resource bundle*.
- Otherwise, `getBundle` attempts to locate a property resource file using the generated properties file name. It generates a path name from the candidate bundle name by replacing all "." characters with "/" and appending the string ".properties". It attempts to find a "resource" with this name using `ClassLoader.getResource`. (Note that a "resource" in the sense of `getResource` has nothing to do with the contents of a resource bundle, it is just a container of data, such as a file.) If it finds a "resource", it attempts to create a new `PropertyResourceBundle` instance from its contents. If successful, this instance becomes the *result resource bundle*.

This continues until a result resource bundle is instantiated or the list of candidate bundle names is exhausted. If no matching resource bundle is found, the default control's `getFallbackLocale` method is called, which returns the current default locale. A new sequence of candidate locale names is generated using this locale and searched again, as above.

If still no result bundle is found, the base name alone is looked up. If this still fails, a `MissingResourceException` is thrown.

Once a result resource bundle has been found, its *parent chain* is instantiated. If the result bundle already has a parent (perhaps because it was returned from a cache) the chain is complete.

Otherwise, `getBundle` examines the remainder of the candidate locale list that was used during the pass that generated the result resource bundle. (As before, candidate bundle names where the final component is an empty string are omitted.) When it comes to the end of the candidate list, it tries the plain bundle name. With each of the candidate bundle names it attempts to instantiate a resource bundle (first looking for a class and then a properties file, as described above).

Whenever it succeeds, it calls the previously instantiated resource bundle's `setParent` method with the new resource bundle. This continues until the list of names is exhausted or the current bundle already has a non-null parent.

Once the parent chain is complete, the bundle is returned.

Note: `getBundle` caches instantiated resource bundles and might return the same resource bundle instance multiple times.

Note: The `baseName` argument should be a fully qualified class name. However, for compatibility with earlier versions, Java SE Runtime Environments do not verify this, and so it is possible to access `PropertyResourceBundle`s by specifying a path name (using "/") instead of a fully qualified class name (using ".").

Example:

The following class and property files are provided:

- MyResources.class
- MyResources.properties
- MyResources_fr.properties
- MyResources_fr_CH.class
- MyResources_fr_CH.properties
- MyResources_en.properties
- MyResources_es_ES.class

The contents of all files are valid (that is, public non-abstract subclasses of `ResourceBundle` for the ".class" files, syntactically correct ".properties" files). The default locale is `Locale("en", "GB")`.

Calling `getBundle` with the locale arguments below will instantiate resource bundles as follows:

Locale	Resource bundle
Locale("fr", "CH")	MyResources_fr_CH.class, parent MyResources_fr.properties, parent MyResources.class
Locale("fr", "FR")	MyResources_fr.properties, parent MyResources.class
Locale("de", "DE")	MyResources_en.properties, parent MyResources.class
Locale("en", "US")	MyResources_en.properties, parent MyResources.class
Locale("es", "ES")	MyResources_es_ES.class, parent MyResources.class

The file MyResources_fr_CH.properties is never used because it is hidden by the MyResources_fr_CH.class. Likewise, MyResources.properties is also hidden by MyResources.class.

API Note:

If the caller module is a named module and the given `loader` is the caller module's class loader, this method is equivalent to `getBundle(baseName, locale)`; otherwise, it may not find resource bundles from named modules. Use `getBundle(String, Locale, Module)` to load resource bundles on behalf on a specific module instead.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`locale` - the locale for which a resource bundle is desired

`loader` - the class loader from which to load the resource bundle

Returns:

a resource bundle for the given base name and locale

Throws:

`NullPointerException` - if `baseName`, `locale`, or `loader` is `null`

`MissingResourceException` - if no resource bundle for the specified base name can be found

Since:

1.2

See Also:

Resource Bundles and Named Modules

getBundle

```
public static ResourceBundle getBundle
(String baseName,
 Locale targetLocale,
 ClassLoader loader,
 ResourceBundle.Control control)
```

Returns a resource bundle using the specified base name, target locale, class loader and control. Unlike the `getBundle` factory methods with no `control` argument, the given `control` specifies how to locate and instantiate resource bundles. Conceptually, the bundle loading process with the given `control` is performed in the following steps.

1. This factory method looks up the resource bundle in the cache for the specified `baseName`, `targetLocale` and `loader`. If the requested resource bundle instance is found in the cache and the time-to-live periods of the instance and all of its parent instances have not expired, the instance is returned to the caller. Otherwise, this factory method proceeds with the loading process below.
2. The `control.getFormats` method is called to get resource bundle formats to produce bundle or resource names. The strings `"java.class"`

and "java.properties" designate class-based and property-based resource bundles, respectively. Other strings starting with "java." are reserved for future extensions and must not be used for application-defined formats. Other strings designate application-defined formats.

3. The control.getCandidateLocales method is called with the target locale to get a list of *candidate Locales* for which resource bundles are searched.
4. The control.newBundle method is called to instantiate a ResourceBundle for the base bundle name, a candidate locale, and a format. (Refer to the note on the cache lookup below.) This step is iterated over all combinations of the candidate locales and formats until the newBundle method returns a ResourceBundle instance or the iteration has used up all the combinations. For example, if the candidate locales are Locale("de", "DE"), Locale("de") and Locale("") and the formats are "java.class" and "java.properties", then the following is the sequence of locale-format combinations to be used to call control.newBundle.

Index	Locale	format
1	Locale("de", "DE")	java.class
2	Locale("de", "DE")	java.properties
3	Locale("de")	java.class
4	Locale("de")	java.properties
5	Locale("")	java.class
6	Locale("")	java.properties

5. If the previous step has found no resource bundle, proceed to Step 6. If a bundle has been found that is a base bundle (a bundle for Locale("")), and the candidate locale list only contained Locale(""), return the bundle to the caller. If a bundle has been found that is a base bundle, but the candidate locale list contained locales other than Locale(""), put the bundle on hold and proceed to Step 6. If a bundle has been found that is not a base bundle, proceed to Step 7.
6. The control.getFallbackLocale method is called to get a fallback locale (alternative to the current target locale) to try further finding a resource bundle. If the method returns a non-null locale, it becomes the next target locale and the loading process starts over from Step 3. Otherwise, if a base bundle was found and put on hold in a previous Step 5, it is returned to the caller now. Otherwise, a MissingResourceException is thrown.
7. At this point, we have found a resource bundle that's not the base bundle. If this bundle set its parent during its instantiation, it is returned

to the caller. Otherwise, its parent chain is instantiated based on the list of candidate locales from which it was found. Finally, the bundle is returned to the caller.

During the resource bundle loading process above, this factory method looks up the cache before calling the `control.newBundle` method. If the time-to-live period of the resource bundle found in the cache has expired, the factory method calls the `control.needsReload` method to determine whether the resource bundle needs to be reloaded. If reloading is required, the factory method calls `control.newBundle` to reload the resource bundle. If `control.newBundle` returns `null`, the factory method puts a dummy resource bundle in the cache as a mark of nonexistent resource bundles in order to avoid lookup overhead for subsequent requests. Such dummy resource bundles are under the same expiration control as specified by `control`.

All resource bundles loaded are cached by default. Refer to `control.getTimeToLive` for details.

The following is an example of the bundle loading process with the default `ResourceBundle.Control` implementation.

Conditions:

- Base bundle name: `foo.bar.Messages`
- Requested `Locale`: `Locale.ITALY`
- Default `Locale`: `Locale.FRENCH`
- Available resource bundles: `foo/bar/Messages_fr.properties` and `foo/bar/Messages.properties`

First, `getBundle` tries loading a resource bundle in the following sequence.

- class `foo.bar.Messages_it_IT`
- file `foo/bar/Messages_it_IT.properties`
- class `foo.bar.Messages_it`
- file `foo/bar/Messages_it.properties`
- class `foo.bar.Messages`
- file `foo/bar/Messages.properties`

At this point, `getBundle` finds `foo/bar/Messages.properties`, which is put on hold because it's the base bundle. `getBundle` calls `control.getFallbackLocale("foo.bar.Messages", Locale.ITALY)` which returns `Locale.FRENCH`. Next, `getBundle` tries loading a bundle in the following sequence.

- class `foo.bar.Messages_fr`
- file `foo/bar/Messages_fr.properties`
- class `foo.bar.Messages`
- file `foo/bar/Messages.properties`

`getBundle` finds `foo/bar/Messages_fr.properties` and creates a `ResourceBundle` instance. Then, `getBundle` sets up its parent chain from the list of the candidate locales. Only `foo/bar/Messages.properties` is found in the list and `getBundle` creates a `ResourceBundle` instance that becomes the parent of the instance for `foo/bar/Messages_fr.properties`.

Parameters:

`baseName` - the base name of the resource bundle, a fully qualified class name

`targetLocale` - the locale for which a resource bundle is desired

`loader` - the class loader from which to load the resource bundle

`control` - the control which gives information for the resource bundle loading process

Returns:

a resource bundle for the given base name and locale

Throws:

`NullPointerException` - if `baseName`, `targetLocale`, `loader`, or `control` is `null`

`MissingResourceException` - if no resource bundle for the specified base name can be found

`IllegalArgumentException` - if the given `control` doesn't perform properly (e.g., `control.getCandidateLocales` returns null.) Note that validation of `control` is performed as needed.

`UnsupportedOperationException` - if this method is called in a named module

Since:

1.6

clearCache

```
public static final void clearCache()
```

Removes all resource bundles from the cache that have been loaded by the caller's module.

Since:

1.6

See Also:

`ResourceBundle.Control.getTimeToLive(String,Locale)`

clearCache

`public static final void clearCache(ClassLoader loader)`

Removes all resource bundles from the cache that have been loaded by the given class loader.

Parameters:

`loader` - the class loader

Throws:

`NullPointerException` - if `loader` is null

Since:

1.6

See Also:

`ResourceBundle.Control.getTimeToLive(String,Locale)`

handleGetObject

`protected abstract Object handleGetObject(String key)`

Gets an object for the given key from this resource bundle. Returns null if this resource bundle does not contain an object for the given key.

Parameters:

`key` - the key for the desired object

Returns:

the object for the given key, or null

Throws:

`NullPointerException` - if `key` is `null`

getKeys

`public abstract Enumeration<String> getKeys()`

Returns an enumeration of the keys.

Returns:

an `Enumeration` of the keys contained in this `ResourceBundle` and its parent bundles.

containsKey

`public boolean containsKey(String key)`

Determines whether the given `key` is contained in this `ResourceBundle` or its parent bundles.

Parameters:

key - the resource key

Returns:

`true` if the given `key` is contained in this `ResourceBundle` or its parent bundles; `false` otherwise.

Throws:

`NullPointerException` - if `key` is `null`

Since:

1.6

keySet

```
public Set<String> keySet()
```

Returns a `Set` of all keys contained in this `ResourceBundle` and its parent bundles.

Returns:

a `Set` of all keys contained in this `ResourceBundle` and its parent bundles.

Since:

1.6

handleKeySet

```
protected Set<String> handleKeySet()
```

Returns a `Set` of the keys contained *only* in this `ResourceBundle`.

The default implementation returns a `Set` of the keys returned by the `getKeys` method except for the ones for which the `handleGetObject` method returns `null`. Once the `Set` has been created, the value is kept in this `ResourceBundle` in order to avoid producing the same `Set` in subsequent calls. Subclasses can override this method for faster handling.

Returns:

a `Set` of the keys contained only in this `ResourceBundle`

Since:

1.6

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.